

UNIVERSIDAD ECOTEC

Actividad Integradora 2

Grano & Co.: reingeniería de la app de catálogo Café Aroma con servicios nativos en Flutter

NOTA: este trabajo **SÍ utiliza el proyecto de la Actividad Integradora 1** (app *Café Aroma*, repositorio `EDWJOSH/cafeteria_app`). Sobre ese mismo proyecto y repositorio se aplicaron todas las mejoras de esta actividad: rebranding, servicios nativos y gestión de ramas.

Estudiante	Edwin Quiroz
Asignatura	Desarrollo de Aplicaciones Móviles
Docente	Gregorio Manuel Cruz Gonzabay
Fecha de entrega	31 de octubre de 2026
Repositorio	https://github.com/EDWJOSH/cafeteria_app

Sección I. Justificación del rebranding y cuadro comparativo

Propuesta de valor

Café Aroma era un catálogo genérico de cafetería: nombre común, ícono por defecto de Flutter y una paleta generada automáticamente desde `Colors.brown`. En la versión 2.0 la app pasa a ser **Grano & Co.**, una cafetería de **café de especialidad tostado en el local**. La nueva propuesta de valor es *"café de especialidad, grano a grano"*: se vende el origen y el tostado propio, no solo una bebida. El nombre corto, con el "& Co.", suena a marca de barrio con trayectoria, y el grano de café funciona como símbolo único que se reconoce en todos los tamaños, desde el splash hasta el ícono de la barra de notificaciones.

Además del cambio visual, la app se modernizó para que funcione como un producto real: descarga el menú desde internet, funciona **sin conexión** con el último menú guardado, permite que el cliente agregue su **foto de perfil** y **avisa con notificaciones** cuando confirma un pedido o cuando cambia la conexión.

Decisiones de identidad visual



Terracota #B5543C
Primario: botones, precios

Espresso #3B2A22
AppBar y textos de título

Crema #F3E6D3
Fondo de pantallas y splash

Oliva #7A8450
Acento: categorías, sello

- **Terracota** recuerda al grano tostado y al barro de las tazas artesanales; es el color de acción (botones, precios).
- **Espresso** en el AppBar da contraste alto para leer el título; **crema** como fondo es más cálido y descansa la vista frente al blanco puro; **oliva** aporta un toque natural para categorías y el sello "Tostado local".
- Títulos en **serif** (Noto Serif Display, licencia libre OFL) para transmitir lo artesanal y premium; el cuerpo de texto sigue en Roboto para que sea legible.
- Todo el estilo vive en un solo archivo, `lib/core/theme/app_theme.dart` (clases `AppColors` y `AppTheme`). Así la paleta es única en toda la app y cambiarla en el futuro es tocar un solo lugar.
- El ícono se generó con `flutter_launcher_icons` (ícono adaptativo en Android: grano crema sobre fondo terracota) y el splash nativo con `flutter_native_splash` (incluye la API de splash de Android 12+).

Cuadro comparativo (antes vs. después)

Elemento	Antes — Café Aroma (v1.0)	Después — Grano & Co. (v2.0)
Nombre comercial	Café Aroma (en el launcher: "cafeteria_app")	Grano & Co.
Package name (Android)	com.edwinquiroz.cafeteria_app	com.edwinquiroz.granoco
Bundle identifier (iOS)	No existía la plataforma iOS	com.edwinquiroz.granoco (se agregó la carpeta ios/)
Paquete Dart / versión	cafeteria_app · 1.0.0+1	grano_co · 2.0.0+2
App Icon	Ícono por defecto de Flutter	Grano de café crema sobre terracota (adaptativo)

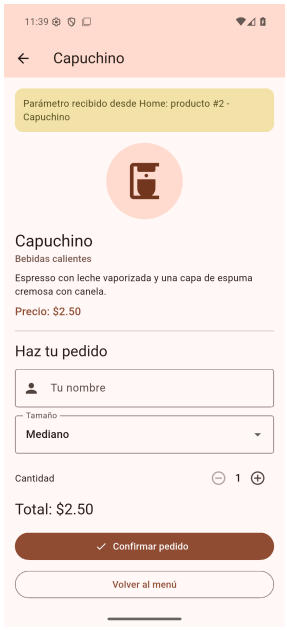
Elemento	Antes — Café Aroma (v1.0)	Después — Grano & Co. (v2.0)
Splash Screen	Pantalla blanca por defecto	Fondo crema, grano terracota y wordmark “Grano & Co. · Café de especialidad”
Paleta	Generada desde Colors.brown (tonos rosados)	Terracota, espresso, crema y oliva definidos a mano
Tipografía	Roboto en todo	Serif de marca en títulos + Roboto en el cuerpo
Banner del Home	Taza y texto “Café Aroma”	Granos de café, “Grano & Co.” y sello “Tostado local”
Pantallas	Home y Detalle	Home, Detalle y Mi perfil (nueva)



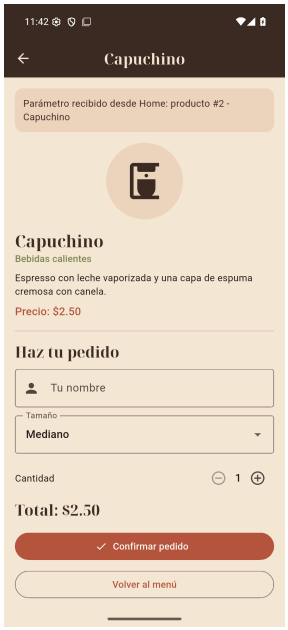
Home — antes



Home — después



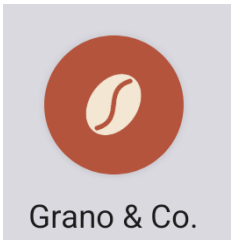
Detalle — antes



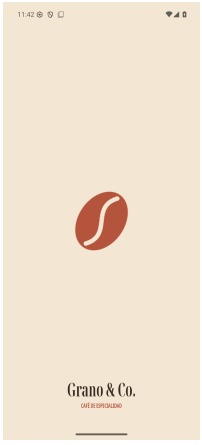
Detalle — después



Ícono — antes



Ícono — después



Splash — después (antes: pantalla blanca)

Capturas tomadas en el emulador Pixel_Cafe (Android 16) antes de empezar los cambios y al terminar el rebranding.

Sección II. Arquitectura y patrones de desarrollo

La app se reorganizó en **capas**. La regla principal es que **la interfaz nunca llama directamente a un plugin nativo**: las pantallas solo leen el estado de los **providers** (patrón *Provider* con `ChangeNotifier`), y los providers usan **servicios** que envuelven a cada plugin (red, cámara, notificaciones, caché). Los servicios se crean una sola vez en `main.dart` y se **inyectan**; por eso en las pruebas se pueden cambiar por servicios falsos.

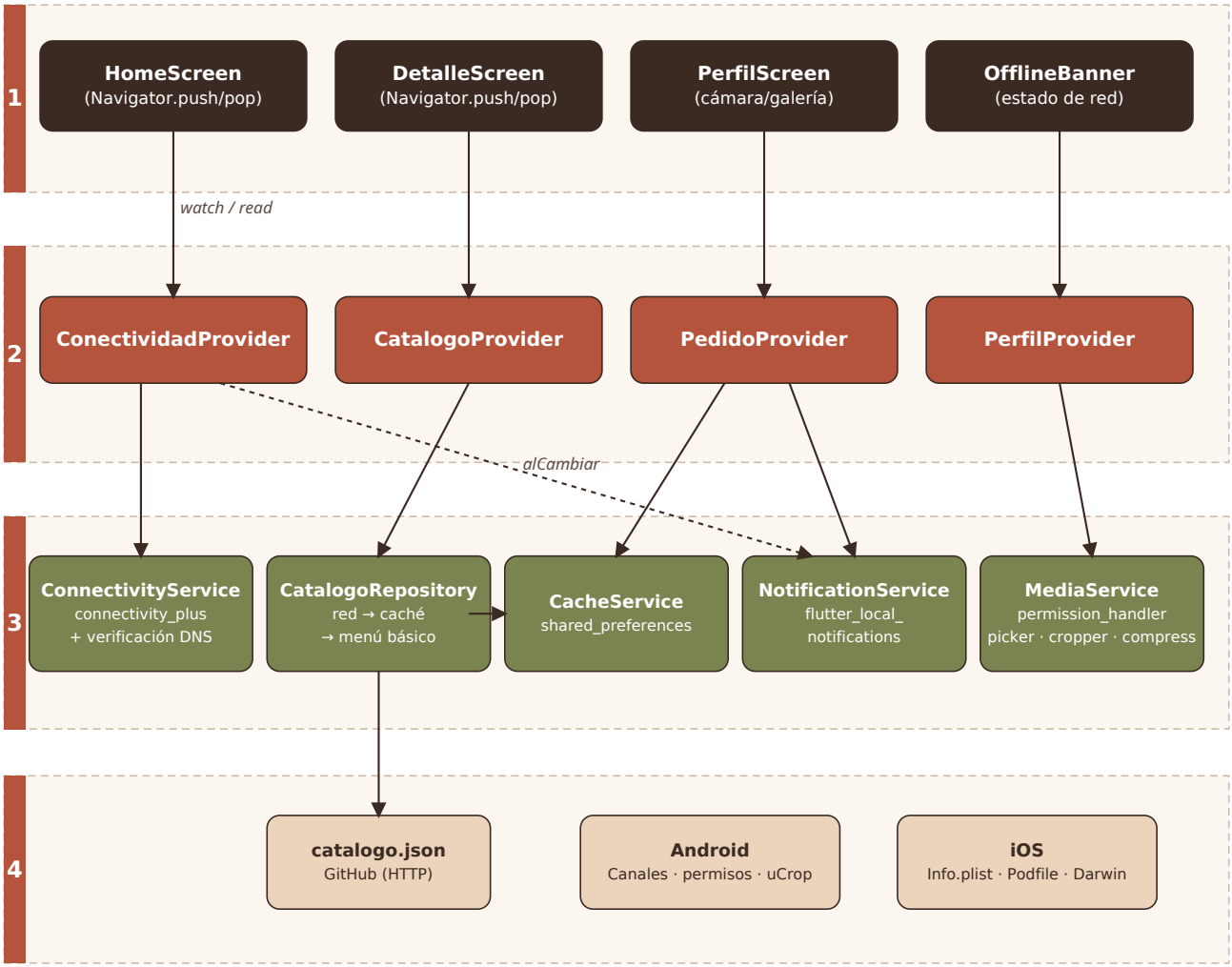


*Figura 1. Diagrama de arquitectura de Grano & Co. Capas: **1** Presentación (screens / widgets), **2** Gestión de estado (Provider + ChangeNotifier, registrados con MultiProvider en main.dart), **3** Servicios nativos y datos (no importan nada de la interfaz) y **4** Plataforma y fuentes externas. Las flechas indican quién usa a quién, siempre de arriba hacia abajo. La flecha punteada es el callback alCambiar que conecta el cambio de red con las notificaciones. Además, un ChangeNotifierProxyProvider hace que CatalogoProvider escuche a ConectividadProvider y recargue el menú al reconectar.*

Patrones aplicados

Patrón / decisión	Dónde	Para qué sirve
Provider (ChangeNotifier)	lib/providers/	Guardar el estado y avisar a la interfaz con notifyListeners(); las pantallas usan context.watch / context.read.
Inyección de dependencias	main.dart (clase Servicios)	Los servicios nativos se crean una vez y se pasan a los providers; en las pruebas se reemplazan por falsos.
Repository	data/catalogo_repository.dart	Ocultar de dónde vienen los datos: primero la red, si falla la caché y, si no hay nada, el menú básico.

Patrón / decisión	Dónde	Para qué sirve
Offline-first + caché	services/cache_service.dart	Guardar el último catálogo, el último pedido y la ruta de la foto con shared_preferences.
Observer (Streams)	ConnectivityService.cambios	El sistema operativo empuja los cambios de red; la app reacciona sin tener que preguntar cada cierto tiempo.
ProxyProvider	main.dart	CatalogoProvider depende de ConectividadProvider: al volver internet recarga el menú solo.

Flujo de un cambio de red (ejemplo)

- Android avisa que se cayó el Wi-Fi → `ConnectivityService.cambios` emite `EstadoRed.sinRed`.
- `ConectividadProvider._actualizar` guarda el estado, llama a `notifyListeners()` y dispara el callback `alCambiar`.
- El `OfflineBanner` se redibuja y aparece; `NotificationService.conexionPerdida()` muestra la notificación del canal *Estado de conexión*.
- Cuando vuelve internet, el `ChangeNotifierProxyProvider` llama a `CatalogoProvider.actualizarConexion`, que descarga el menú de nuevo.

Sección III. Explicación de código y manejo de excepciones

III.1 Detección reactiva de conectividad

Tener Wi-Fi no significa tener internet (por ejemplo, una red de hotel sin salida). Por eso el servicio hace dos comprobaciones: primero si hay alguna interfaz de red y después si se puede resolver un dominio real por DNS.

lib/services/connectivity_service.dart (líneas 17–44) · Stream de cambios y verificación real de internet

```
/// Emite un nuevo estado cada vez que el sistema operativo avisa un cambio de red.
Stream<EstadoRed> get cambios =>
    _connectivity.onConnectivityChanged.asyncMap(_evaluar);

Future<EstadoRed> estadoActual() async =>
    _evaluar(await _connectivity.checkConnectivity());

Future<EstadoRed> _evaluar(List<ConnectivityResult> resultados) async {
  final sinInterfaz =
      resultados.isEmpty ||
      resultados.every((r) => r == ConnectivityResult.none);
  if (sinInterfaz) return EstadoRed.sinRed;
  return await hayInternet() ? EstadoRed.enLinea : EstadoRed.sinInternet;
}

/// Comprueba salida real a internet resolviendo un dominio (DNS).
Future<bool> hayInternet() async {
  try {
    final resultado = await InternetAddress.lookup(
      'raw.githubusercontent.com',
    ).timeout(const Duration(seconds: 4));
    return resultado.isNotEmpty && resultado.first.rawAddress.isNotEmpty;
  } on SocketException {
    return false;
  } on TimeoutException {
    return false;
  }
}
```

- `onConnectivityChanged` es un **Stream** que el plugin `connectivity_plus` alimenta desde el sistema operativo. Con `asyncMap(_evaluar)` cada evento se convierte en uno de los tres estados del enum `EstadoRed`.
- Si la lista viene vacía o todo es `ConnectivityResult.none`, el teléfono no tiene red: `sinRed`.
- `hayInternet()` intenta `InternetAddress.lookup` con un límite de 4 segundos (`.timeout`).
- **Excepciones:** si no hay salida, Dart lanza `SocketException`; si la red es lenta, `TimeoutException`. Las dos se capturan con `on ... { return false; }`: en vez de cerrarse, la app concluye *"sin internet"*.

lib/providers/conectividad_provider.dart (líneas 27–43) · Provider que escucha el Stream

```
Future<void> _iniciar() async {
  _actualizar(await _servicio.estadoActual());
  _suscripcion = _servicio.cambios.listen(_actualizar);
}

Future<void> reintentar() async =>
    _actualizar(await _servicio.estadoActual());

void _actualizar(EstadoRed nuevo) {
  if (_verificado && nuevo == _estado) return;
  final anterior = _estado;
  final primeraLectura = !_verificado;
  _estado = nuevo;
  _verificado = true;
  notifyListeners();
  if (!primeraLectura) alCambiar?.call(anterior, nuevo);
}
```

- `_iniciar` lee el estado actual una vez y después se suscribe al Stream con `listen`. La suscripción se cancela en `dispose()` para no dejar escuchas vivas cuando el provider se destruye.
- `_actualizar` ignora eventos repetidos, avisa a la interfaz con `notifyListeners()` y, salvo en la primera lectura, llama a `alCambiar` (así la app no lanza una notificación al abrirse).

lib/data/catalogo_repository.dart (líneas 30–63) · Estrategia offline-first con fallback

```
Future<ResultadoCatalogo> obtener({required bool enLinea}) async {
  if (enLinea) {
    try {
      final respuesta = await _cliente
          .get(Uri.parse(urlCatalogo))
          .timeout(const Duration(seconds: 8));
      if (respuesta.statusCode == 200) {
        final productos =
            (jsonDecode(utf8.decode(respuesta.bodyBytes)) as List)
                .map(
                  (item) => Producto.fromJson(item as Map<String, dynamic>),
                )
                .toList();
        await _cache.guardarCatalogo(productos);
        return ResultadoCatalogo(
          productos,
          OrigenCatalogo.red,
          DateTime.now(),
        );
      }
    } catch (_) {
      // Si la descarga falla seguimos con la caché (estrategia offline-first).
    }
  }

  final guardado = _cache.leerCatalogo();
  if (guardado != null) {
    return ResultadoCatalogo(
      guardado.productos,
      OrigenCatalogo.cache,
      guardado.fecha,
    );
  }
  return const ResultadoCatalogo(catalogoBase, OrigenCatalogo.local, null);
```

- Con internet descarga `catalogo.json` desde GitHub con límite de 8 segundos y lo guarda en caché.
- **Excepciones:** cualquier error de red, de tiempo o de JSON mal formado cae en `catch (_)` y el método sigue con la caché. Si tampoco hay caché, devuelve `catalogoBase`. La pantalla **nunca queda vacía**.
- El resultado indica el **origen** (`red / cache / local`) y la fecha, que se muestran en el banner y bajo “Nuestro menú”.

III.2 Manejo dinámico de permisos (cámara y galería)

lib/services/media_service.dart (líneas 29–49) · Solicitud del permiso en tiempo de ejecución

```
/// Pide el permiso que corresponde a la fuente elegida.
Future<EstadoPermiso> solicitarPermiso(ImageSource fuente) async {
  final Permission permiso;
  if (fuente == ImageSource.camera) {
    permiso = Permission.camera;
  } else if (Platform.isIOS) {
    permiso = Permission.photos;
  } else {
    // Android 13+ usa el selector de fotos del sistema: no necesita permiso.
    return EstadoPermiso.concedido;
  }

  var estado = await permiso.status;
  if (estado.isGranted || estado.isLimited) return EstadoPermiso.concedido;
  if (estado.isPermanentlyDenied) return EstadoPermiso.denegadoPermanente;

  estado = await permiso.request();
  if (estado.isGranted || estado.isLimited) return EstadoPermiso.concedido;
```

```
  if (estado.isPermanentlyDenied) return EstadoPermiso.denegadoPermanente;
  return EstadoPermiso.denegado;
}
```

- Según la fuente se elige el permiso: `Permission.camera` para la cámara y `Permission.photos` para la galería en iOS. En Android 13+ la galería usa el **selector de fotos del sistema**, que no necesita permiso.
- Primero se consulta `permiso.status` para no volver a preguntar si ya se concedió, y si el permiso está bloqueado no se insiste.
- Si hace falta, `permiso.request()` muestra el diálogo nativo. El resultado se traduce a tres casos: **concedido**, **denegado** (puede volver a preguntarse) y **denegado permanentemente** (Android ya no muestra el diálogo).

lib/screens/perfil_screen.dart (líneas 14–36) · La interfaz decide qué mostrar según el resultado

```
Future<void> _cambiarFoto(BuildContext context, ImageSource fuente) async {
  final perfil = context.read<PerfilProvider>();
  final estado = await perfil.cambiarFoto(fuente);
  if (!context.mounted) return;

  switch (estado) {
    case EstadoPermiso.concedido:
      if (perfil.error != null) {
        ScaffoldMessenger.of(context)
            .showSnackBar(SnackBar(content: Text(perfil.error!)));
      }
    case EstadoPermiso.denegado:
      ScaffoldMessenger.of(context).showSnackBar(
        const SnackBar(
          content: Text(
            'Sin permiso no podemos usar la cámara. Puedes intentarlo otra vez.',
          ),
        ),
      );
    case EstadoPermiso.denegadoPermanente:
      await _mostrarDialogoAjustes(context);
  }
}
```

- Después del `await` se comprueba `context.mounted`: si el usuario salió de la pantalla mientras elegía la foto, no se usa un `context` que ya no existe.
- Denegado → `SnackBar` con una explicación. Denegado permanentemente → `AlertDialog` con el botón **Abrir ajustes** (`openAppSettings()`), porque es la única forma de reactivarlo.

lib/providers/perfil_provider.dart (líneas 31–54) · Captura, recorte y compresión con try / catch / finally

```
Future<EstadoPermiso> cambiarFoto(ImageSource fuente) async {
  final permiso = await _media.solicitarPermiso(fuente);
  if (permiso != EstadoPermiso.concedido) return permiso;

  _procesando = true;
  _error = null;
  notifyListeners();
  try {
    final resultado = await _media.obtenerFoto(fuente);
    if (resultado != null) {
      final anterior = _foto;
      _foto = resultado.archivo;
      _ultimoResultado = resultado;
      await _cache.guardarFotoPerfil(resultado.archivo.path);
      if (anterior != null && anterior.existsSync()) await anterior.delete();
    }
  } catch (e) {
    _error = 'No se pudo procesar la imagen: $e';
  } finally {
    _procesando = false;
    notifyListeners();
  }
  return permiso;
}
```

- `obtenerFoto` usa `image_picker` (captura), `image_cropper` (recorte circular 1:1) y `flutter_image_compress` (JPEG calidad 70, 600 px). Si el usuario cancela devuelve `null` y no cambia nada.
- **Excepciones:** un fallo nativo (cámara ocupada, archivo dañado) cae en `catch (e)` y se guarda en `_error` para mostrarlo; `finally` siempre apaga el indicador de carga, aunque haya error.
- Se borra la foto anterior del disco para no acumular archivos.

III.3 Servicio de notificaciones locales

lib/services/notification_service.dart (líneas 10–50) · Canales de Android e inicialización

```
/// Canal de alta importancia: confirmación de pedidos.
static const canalPedidos = AndroidNotificationChannel(
  'pedidos',
  'Pedidos',
  description: 'Confirmación de los pedidos que haces en la app',
  importance: Importance.high,
);

/// Canal de importancia normal: cambios de conexión.
static const canalConexion = AndroidNotificationChannel(
  'conectividad',
  'Estado de conexión',
  description: 'Avisos cuando se pierde o se recupera internet',
);

final FlutterLocalNotificationsPlugin _plugin =
    FlutterLocalNotificationsPlugin();
int _siguienteId = 0;

bool get _soportado => !kIsWeb && (Platform.isAndroid || Platform.isIOS);

/// Configura el plugin y registra los canales en Android.
Future<void> inicializar() async {
  if (!_soportado) return;
  await _plugin.initialize(
    settings: const InitializationSettings(
      android: AndroidInitializationSettings('ic_notificacion'),
      iOS: DarwinInitializationSettings(
        requestAlertPermission: false,
        requestBadgePermission: false,
        requestSoundPermission: false,
      ),
    ),
  );
  final android = _plugin
      .resolvePlatformSpecificImplementation<
        AndroidFlutterLocalNotificationsPlugin
      >();
  await android?.createNotificationChannel(canalPedidos);
  await android?.createNotificationChannel(canalConexion);
}
```

- Desde Android 8 toda notificación debe pertenecer a un **canal**. Se crearon dos: `pedidos` con `Importance.high` (aparece arriba y suena) y `conectividad` con importancia normal. El usuario puede silenciar uno sin afectar el otro desde los Ajustes.
- `initialize` recibe la configuración de Android (ícono monocromático `ic_notificacion`) y la de iOS (`DarwinInitializationSettings`). En iOS no se pide permiso al iniciar, sino en el momento adecuado.
- `_soportado` evita llamar al plugin en web o en Linux, donde no aplica.

lib/main.dart (líneas 59–66) · Eventos clave conectados al servicio

```
/// Notifica solo los cambios entre "con internet" y "sin internet".
void _notificarCambioDeRed(EstadoRed anterior, EstadoRed nuevo) {
  if (anterior == EstadoRed.enLinea && nuevo != EstadoRed.enLinea) {
    servicios.notificaciones.conexionPerdida();
  } else if (anterior != EstadoRed.enLinea && nuevo == EstadoRed.enLinea) {
    servicios.notificaciones.conexionRecuperada();
  }
}
```

En `solicitarPermiso()` se pide `POST_NOTIFICATIONS` (Android 13+) o los permisos de iOS al abrir la Home. Los eventos clave son: **pedido confirmado** (`PedidoProvider.registrar`), **conexión perdida** y **conexión restablecida** (arriba). Solo se notifica el paso entre *con internet* y *sin internet*, no cada evento del Stream.

III.4 Configuración nativa

Archivo	Cambio	Motivo
AndroidManifest.xml	INTERNET, ACCESS_NETWORK_STATE, CAMERA, POST_NOTIFICATIONS, UCropActivity	Permisos que declara la app y pantalla nativa de recorte
android/app/build.gradle.kts	applicationId com.edwinquiroz.granoco, compileSdk 37, coreLibraryDesugaring	Nuevo identificador; requisitos de permission_handler 13 y flutter_local_notifications
ios/Runner/Info.plist	NSCameraUsageDescription, NSPhotoLibraryUsageDescription	iOS exige explicar al usuario para qué se usa cada permiso
ios/Podfile	PERMISSION_CAMERA, PERMISSION_PHOTOS, PERMISSION_NOTIFICATIONS	permission_handler solo compila los permisos que se usan
ios/Runner/AppDelegate.swift	UNUserNotificationCenter delegate	Mostrar notificaciones con la app abierta en iOS

Sección IV. Evidencias de pruebas (QA) y enlace al repositorio público

Repositorio público: https://github.com/EDWJOSH/cafeteria_app
Ramas: `main` (estable), `develop` (integración) y `feature/rebranding`, `feature/conectividad`, `feature/camara-permisos`, `feature/notificaciones`.

Todas las capturas se tomaron en el emulador **Pixel_Cafe (Android 16, API 36)** con la app compilada desde la rama `develop`.

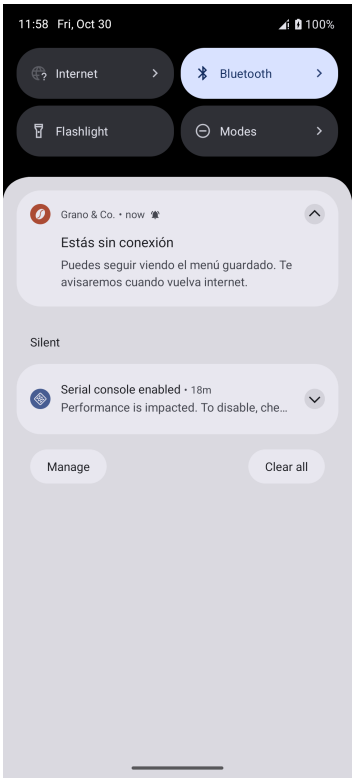
IV.1 Modo offline (conectividad en tiempo real)



1. Con internet: menú descargado (incluye el Cold brew, que solo existe en el JSON remoto).



2. Wi-Fi y datos apagados: banner offline y menú guardado en el equipo.



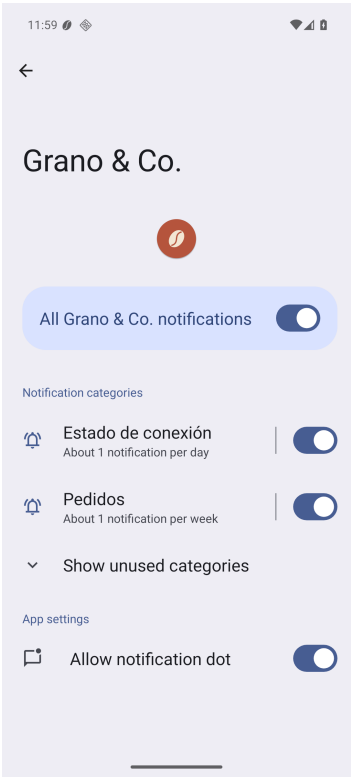
3. Notificación "Estás sin conexión" (canal Estado de conexión).



4. Red restablecida: el banner desaparece y el menú se vuelve a descargar.



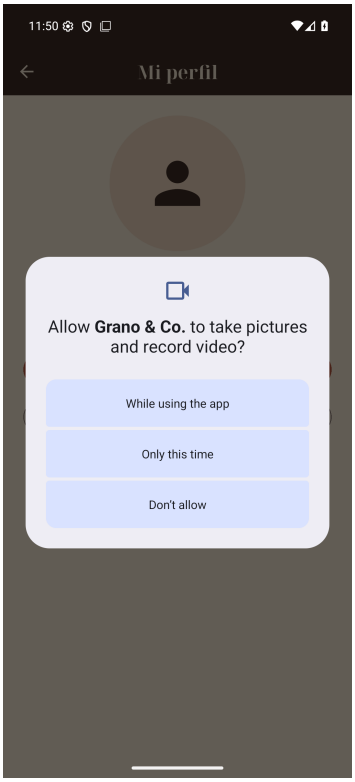
5. Notificación "Conexión restablecida".



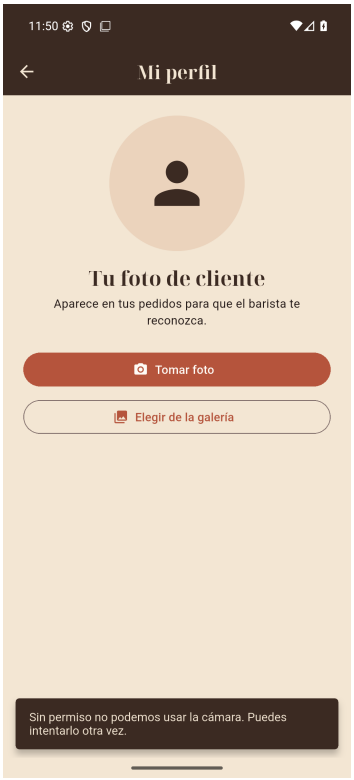
6. Canales registrados en Ajustes: Pedidos y Estado de conexión.

Para cortar la red en el emulador se usó `adb shell svc wifi disable` y `adb shell svc data disable`, y para restablecerla, `enable`. La app reaccionó sola, sin reiniciarse.

IV.2 Solicitud de permisos de cámara



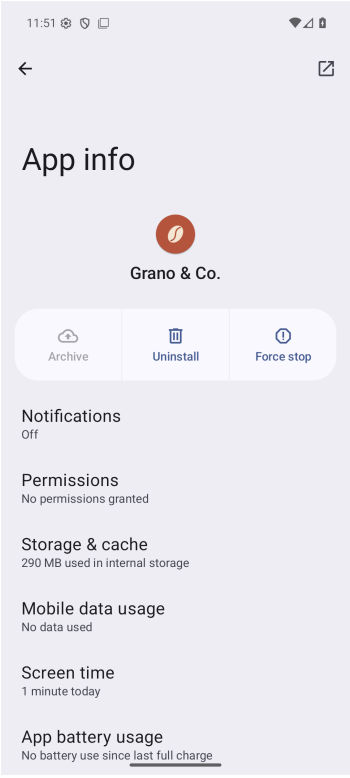
1. Diálogo nativo de Android al tocar "Tomar foto".



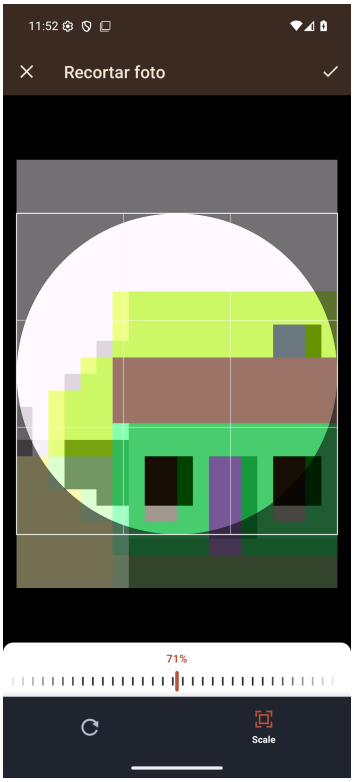
2. Primer rechazo: aviso con SnackBar.



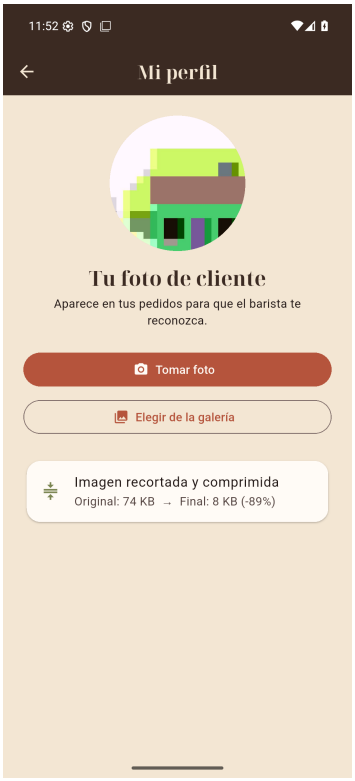
3. Segundo rechazo (bloqueado): diálogo con "Abrir ajustes".



4. "Abrir ajustes" lleva a la información de la app en Android.



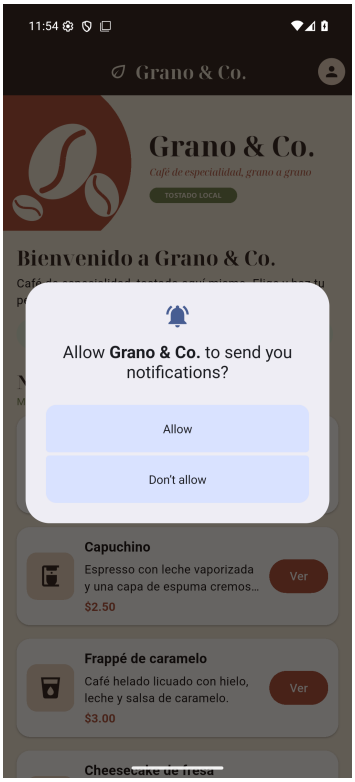
5. Permiso concedido: foto tomada y recorte circular con colores de la marca.



6. Foto guardada: comprimida de 74 KB a 8 KB (-89 %).

La imagen pixelada es la escena virtual que genera la cámara del emulador de Android.

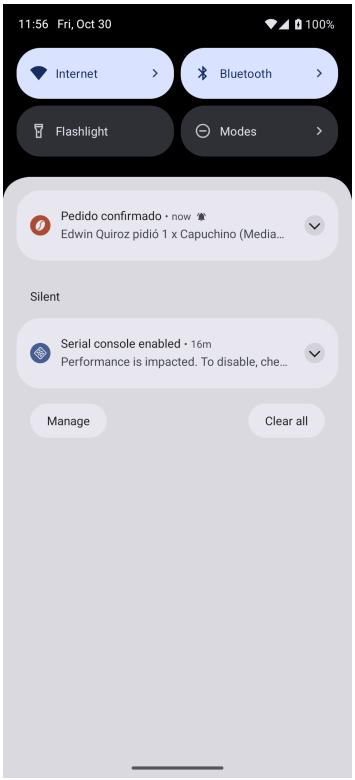
IV.3 Recepción de notificaciones



1. Permiso POST_NOTIFICATIONS (Android 13+) al abrir la app.



2. Pedido de un Capuchino con el nombre del cliente.



3. Notificación "Pedido confirmado" con el ícono de la marca (canal Pedidos).

IV.4 Pruebas automáticas y análisis estático

#	Prueba de widget (flutter test)	Resultado
1	Con internet muestra el menú descargado de la red	✓ Pasó
2	Sin internet muestra el banner offline y el menú básico	✓ Pasó
3	Al perder la red usa la caché y al volver se recarga	✓ Pasó
4	Navega al detalle con el producto y regresa con el pedido	✓ Pasó
5	Permiso de cámara bloqueado: ofrece abrir los ajustes	✓ Pasó
6	Permiso de cámara denegado: muestra un aviso	✓ Pasó
7	Confirmar un pedido dispara la notificación del canal pedidos	✓ Pasó
8	Perder y recuperar internet notifica en el canal conectividad	✓ Pasó

Resultado: **8 de 8 pruebas pasaron** (“All tests passed!”). `flutter analyze`: “No issues found!”. Las pruebas usan servicios falsos (`FakeConnectivityService`, `FakeMediaService`, `FakeNotificationService`, y un `MockClient` de http), posibles gracias al desacoplamiento de la Sección II.

IV.5 Historial de commits y ramas

```
* 7f2f65e [develop] Docs: README profesional de la versión 2.0 (arquitectura, instal…
* fb879e7 Fix: el menú se marca como guardado en el equipo al perder la co…
*   dd0c167 Merge feature/notificaciones en develop
|\
| * 1d4922a [feature/notificaciones] Notificaciones locales: canales Android, permiso y eventos clave
|/
*   794a56f Merge feature/camara-permisos en develop
|\
| * 24c34be [feature/camara-permisos] Cámara y galería: permisos en tiempo de ejecución, recorte y com…
|/
*   3107b7f Merge feature/conectividad en develop
|\
| * 7f823ed [feature/conectividad] Conectividad: detección reactiva de red, banner offline y caché …
|/
*   bed02f7 Merge feature/rebranding en develop
|\
| * ab1f118 [feature/rebranding] Rebranding visual: ícono, splash screen y paleta unificada de Gr…
| * 7a1083c Rebranding: nuevo nombre Grano & Co. e identificador com.edwinqu…
|/
* 7951fbf [main] Ajustes visuales, depuración y actualización de README.md
* 84e8aa3 Implementación de navegación multi-pantalla y paso de parámetros
* 06237ec Creación de la Pantalla 2 (DetalleScreen) y estructura de datos
* 9b77b7d Maquetación de la Pantalla 1 (HomeScreen) con widgets básicos
* 8668e91 Setup inicial: proyecto Flutter, .gitignore y README
```

Las 5 primeras entradas (abajo) son los commits de la Actividad Integradora 1 en `main`. Encima están los **commits nuevos** de esta actividad, cada uno hecho en su rama `feature/*` e integrado en `develop` con `merge --no-ff`, para que el grafo muestre de dónde vino cada cambio. Al cerrar la versión, `develop` se fusiona en `main`.

Sección V. Conclusiones y lecciones aprendidas

Ciclo de vida del dispositivo y de la app

- Un permiso no es un sí o un no: en Android pasa por **no solicitado → denegado → denegado permanentemente**, y el usuario puede cambiarlo desde los Ajustes con la app en segundo plano. Por eso la app consulta `status` cada vez, en lugar de recordar la respuesta.
- La red cambia en cualquier momento. Escuchar un **Stream** en lugar de preguntar una sola vez al inicio hizo que la app reaccione sola a la pérdida y a la vuelta de internet.
- Los recursos nativos hay que liberarlos: la suscripción al Stream se cancela en `dispose()` y la foto anterior se borra del disco.
- Desde Android 13 las notificaciones también requieren permiso en tiempo de ejecución, y desde Android 8 deben ir en canales.

Concurrencia y asincronía

- Casi todo lo nativo es asíncrono (`Future` / `await`). Entre que se pide la foto y que vuelve, el usuario puede salir de la pantalla; comprobar `context.mounted` después de cada `await` evita errores.
- Las llamadas de red llevan `.timeout()` y sus excepciones se capturan, para que una red lenta no congele la app.
- Cambiar el estado durante la construcción de un widget provoca errores en Flutter; por eso la recarga del catálogo se agenda con `Future.microtask`.
- Las pruebas automáticas encontraron un error real de lógica: al perder la red el banner decía “menú básico” aunque se mostraba el descargado. Se corrigió y se agregó la verificación a la prueba.

Control de versiones

- Trabajar con **ramas por funcionalidad** permitió desarrollar cada servicio aislado y tener `main` siempre estable.
- Usar `merge --no-ff` deja un historial que se puede auditar: en el grafo se ve qué commits pertenecen a cada funcionalidad.
- Commits pequeños con mensajes descriptivos facilitan revisar y revertir cambios; por ejemplo, la corrección del banner quedó en un commit propio.
- Mantener el mismo repositorio de la Actividad 1 conserva la trazabilidad completa del proyecto, de la v1.0 a la v2.0.

Posibles mejoras

- Conectar el catálogo y los pedidos a un backend real (por ejemplo Firebase) en lugar de un JSON estático.
- Programar notificaciones (“tu pedido está listo”) y abrir la pantalla correcta al tocar una notificación.
- Agregar pruebas de integración en un dispositivo real y un pipeline de CI en GitHub Actions.